

Contagem de Primitivas na Gestão de Laços Iterativos em Python

Diego Frias

August 2026

1 Introdução

Primitivas O.C.A na gestão de um **laço iterativo simples**:

```
for i in np.arange(x, y, k): => i = [x, x + k, x + 2k, ..., x + (n-1)k]
```

Caso particular:

```
for i in range(n):
```

$$x = 0,$$

$$y = n,$$

$$k = 1$$

$$\Rightarrow i = [0, 1, 2, \dots, n - 1]$$

Passo-a-passo:

1. atribuir valor final $y = n$,	A = 1
2. atribuir valor inicial $x = 0$,	A = 1
3. atribuir valor incremento $k = 1$,	A = 1
4. inicializar contador $i = x$,	A = 1
5. if $i < y$:	C = 1
SIM:	
6. $i = i + k$, volta ao passo 5	A = 1, O = 1
NÃO:	
7. Exit	

RESUMO :

- INICIALIZAÇÃO DO LOOP (g_1): passos 1 a 4: A = 4,

$$g1 = (0, 0, 0, 4)$$

- A CADA ITERAÇÃO/REPETIÇÃO (g_r): passo 5 + (passo 6 OU passo 7).

Como o custo do passo 7 não está no FOCA, apenas precisamos considerar o passo 5 + passo 6, então

$$g_r = (0, 1, 1, 1)$$

O FOCA do laço vem dado então por:

$$FOCA_{loop} = g_1 + Xg_r$$

onde X é o número de vezes que os passos 5 e 6 são executados!

Mas, nós sabemos que

$$X = 1 + \frac{\max(x, y) - \min(x, y)}{\text{abs}(k)}$$

pelo que:

$$X = 1 + \frac{\max(0, n) - \min(0, n)}{\text{abs}(1)}$$

$$X = 1 + \frac{n - 0}{1} = n + 1$$

Então o

$$FOCA_{loop} = g_1 + (n + 1)g_r$$

Substituindo os vetores g_1 e g_r obtemos:

$$FOCA_{loop} = (0, 0, 0, 4) + (n + 1)(0, 1, 1, 1)$$

Fazendo a operação de produto de constante por vetor obtemos:

$$FOCA_{loop} = (0, 0, 0, 4) + (0, (n + 1), (n + 1), (n + 1))$$

Fazendo agora a operação soma de vetores obtemos:

$$FOCA_{loop} = (0, (n + 1), (n + 1), (n + 1) + 4)$$

1.1 Abordagem Simplificada

Desconsiderando que as primitivas têm tempos distintos podemos obter o número total de primitivas na gestão de laços P_{loop} em função de n , ou seja

$$P_{loop}(n) = O(n) + C(n) + A(n)$$

$$P_{loop}(n) = (n + 1) + (n + 1) + (n + 1) + 4$$

$$P_{loop}(n) = 7 + 3n$$

1.2 Abordagem Detalhada

Podemos estimar o tempo gasto na gestão do laço T_{loop} se multiplicarmos o número de cada primitiva pelo tempo médio de cada uma, ou seja, por τ_o, τ_c, τ_a :

$$T_{loop}(n) = O(n) \tau_o + C(n) \tau_c + A(n) \tau_a$$

Substituindo O, C, A temos

$$T_{loop}(n) = (n + 1) \tau_o + (n + 1) \tau_c + (n + 5) \tau_a$$

Contudo, para isso precisamos estimar experimentalmente τ_o, τ_c, τ_a no ambiente (hardware + SO + versão do Python) em que será executado o programa em questão.